

Speeding up WPEWebKit and WebKitGTK with Profile- Guided Optimization

Web Engines Hackfest
June 2026, A Coruña

Lauro Moura

About me

- At Igalia since 2020
- WebKit team
 - QA, WebDriver
- Other work done in the past:
 - EFL (JS and C# bindings)
 - Maemo (PySide Python bindings, Nix WebKit port)



Agenda

- Motivation
- PGO 101
- Profile matters
- PGO in WebKit
- Test results
- Future work



Motivation

- Browsers are **huge pieces of software**
 - Also running large, complex software
 - Running as **fast as possible** is a must
- **Compiler heuristics** help a lot, but **they're not crystal balls** 🧙‍♂️
 - *Is it for a long-running totem, with little to no interaction?*
 - *Or is it a fully interactive browser, that will open and close frequently?*
- What if the **compiler** could actually **see how the program will run?**



Enter Profile-Guided Optimization (PGO)

- aka Feedback-Directed Optimization - FDO
- Provide the compiler information about **actual software behavior**
- Helps decisions like **branch prediction, inlining**, etc
- It's a *relatively* **low-hanging** fruit to speed up software
 - Although it comes with its own share of **caveats** 🗨️🗨️



PGO 101

- Compiler: *"What do you want me to optimize for?"*
 - Kinda like a `-Owhat`
- Step #1: **Collect profiling data**
 - Running **representative** workloads
- Step #2: **Optimized build** analyzes the collected data
 - Improved performance 🙌



Profile sources

- Instrumented builds
 - Compiler Frontend-based
 - Better source correlation, but more expensive. Ideally for test coverage
 - **IR-based**
 - Lower overhead, recommended by `clang` for PGO builds
- Sampled data (e.g. `perf`)
 - Even lower overhead, but relies on statistical snapshots



Instrumented builds

- Injected code by the compiler into the final binary
- Adds some overhead, but gives more precise coverage
- **Partial instrumentation** is possible, like **specific functions or files**
- `__llvm_profile_dump()` writes the collected data, typically in `atexit` handlers
 - ⚠ Could be a problem in **abnormal terminations**, losing collected data



Instrumented profile data

- **Compiler-specific** formats (i.e. can't mix `gcc` and `clang` instrumented data)
- `default.profraw` files, usually one per process
 - Can be overridden with `LLVM_PROFILE_FILE` envvar, adding the PID with `%p`
- `llvm-profdata merge -output motionmark.profdata *.profraw`



Instrumented profile data (cont.)

- `llvm-profdata-18 show --topn=20 profdata.profdata`

```
Instrumentation level: IR  entry_first = 0  instrument_loop_entries  
Total functions: 253015  
Maximum function count: 157512540  
Maximum internal block count: 86383595  
Total number of blocks: 2542631  
Total count: 20154012793
```



Instrumented profile data (cont.)

- `llvm-profdata-18 show --topn=20 profdata.profdata`

Top 10 functions with the largest internal block counts:

JSC::CodeOrigin::CodeOrigin(JSC::CodeOrigin const&), max count =

WTF::fastFree(void*), max count = 86383595

pas_thread_local_cache_flush_deallocation_log, max count = 85485

WTF::FastBitVector::at(unsigned long), max count = 75859569

<snip>



Profile workload selection

- Should be **representative** of the actual user workload
-  **Beware of using unit tests**, as they often focus on exceptional paths
 - Integration and replay tests are better
- Different workload profiles **can be merged** with **different weights**
 - `Motionmark* 0.6 + Speedometer*0.2 + JetStream*0.2`



Other things to keep in mind

- PGO in Yocto builds
 - Profile data must be tracked properly, to avoid **SSTATE poisoning and false cache hits**
 - For example, **adding the profile file as a SRC_URI**
- **ccache** false hits
 - By default, **profile content is not hashed** by **ccache**
 - We can use **CCACHE_EXTRAFILES=/path/to/profdata** to make it explicit, or **version the filename**
- Beware of relying on PGO **too much**
 - For example, an eventual PGO variation **hiding an actual regression**



PGO in WebKit



PGO in WPEWebKit

- Until recently, supported only by Apple ports (XCode builds)
- CMake support landed in **April 2026** for WPEWebKit and WebKitGTK ([310954@main](#))
- IR-level instrumented builds with **clang**
- Instrumentation builds: `-DENABLE_LLVM_PROFILE_GENERATION=1`
 - `-fprofile-generate`
- Optimized builds: `-DUSE_PGO_PROFILE=1` and `PGO_PROFILE_PATH=target.profdata`
 - `-fprofile-use=${PGO_PROFILE_PATH}`



PGO in WPEWebKit - LTO issues

- Instrumented code exposed `__llvm_profile_name` through multiple translation units
 - `-Wl,--allow-multiple-definitions`



PGO in WPEWebKit - Profile Counter Mismatches

- Source changes between instrumented and optimized build can trigger `backend-plugin` warnings
 - Instrumentation-specific handlers, for example
 - `-Wno-error-backend-plugin`



PGO in WPEWebKit - Clean shutdown

- WebKit apps have multiple child processes: `WebProcess`, `NetworkProcess`
- Usually terminated through `SIGKILL` during shutdown
- Instrumented-build changes:
 - UIProcess (browser) will try `SIGTERM` before `SIGKILL`
 - Child processes: explicitly call `__llvm_profile_dump()` in `SIGTERM` handlers
 - **Disable sandboxing**, as `bwrap` interfered with the graceful termination



PGO in WPEWebKit - icecc issues

- Instrumented builds: `icecc` thinks `-fprofile-generate` will generate artifacts, **forcing local builds**
 - Passing a path as `-fprofile-generate=/foo/bar` bypasses it.
 - For example, the expected `LLVM_PROFILE_FILE` mask (e.g. `/tmp/profile-%p.profraw`)
- Optimised builds: `icecc` requires the `-fprofile-use=` argument to exist locally.
 - `ICECC_EXTRAFILES` did not work for the profiles, though
 - So, currently, **optimized builds are done locally**



Tests results



Test environment

- Machine: x86-64 Ubuntu laptop
- PGO patch on top of 307824@main (Mid Feb 2026)
- N=2, **not statistically robust**, originally for guidance during development
- Configurations:
 - Vanilla WPE MiniBrowser
 - Instrumented WPE MiniBrowser
 - LTO-only build
 - Optimized WPE MiniBrowser with training from:
 - Jetstream
 - Speedometer
 - Motionmark
 - **Merged profiles**



Results

- **Instrumented builds** run noticeably **slower as expected** (-10% ~ -30%)
- **LTO** alone: **small-to-moderate gains** over vanilla. ($\leq 10\%$)
- **PGO on top of LTO**: pointed further up **in every case we tried**, roughly on the order of +5–15% depending on benchmark.
- **Training on the same benchmark you measure** gives the best numbers, **but might be overfitting** for the real world
- **Merged-profile** training (the **realistic** option) captured **most of the gains** of the matched-profile scenarios



Future work



Future work

- Add PGO support to WPEWebKit and WebKitGTK CI
 - Properly track performance improvements
 - Motionmark, Jetstream, Speedometer
 - Epiphany PGO flatpak?
- Measure PGO improvements on embedded boards (e.g. rpi4)
- Explore partial instead of full instrumentation
 - Reduce overhead and fine-tune gains
- Explore sample-based PGO



Key Takeaways

- PGO helps the compiler take better informed decisions about what it should optimize for
 - Initial WebKit results are promising
- But it's not a silver bullet
 - Selecting a representative workload is essential for good results



Questions?



Thanks!



